

R functions for repSpat

Loading libraries

```
library(sf)
library(ggplot2)
library(spdep)
library(Matrix)
library(MASS)
library(dplyr)
library(sp)
library(openxlsx)
library(adespatial)
library(igraph)
library(mclust)
library(SpatialExperiment)
library(Banksy)
library(scater)
```

Extract data for each tissue from spatilExperiment object

The output of `get_sample_data` are the data frame with intensities/binary, spatial coordinates, and distance matrix.

Marker intensities

```
get_sample_data <- function(spe, sample_number){

  # Sub-setting patient data
  spe_patient <- spe[, spe$sample_id==paste("Sample",sample_number, sep="_")]
```

```

# Creating the feature and coordinates matrix for the selected patient
feature_mat <- as.data.frame(t(assay(spe_patient)))
coords_mat <- as.data.frame(spatialCoords(spe_patient))

# Create a data frame using features and coordinates
patient_data <- cbind(feature_mat, spatialCoords(spe_patient), cells = spe_patient$mm)

# Removing gene names from the row header
rownames(feature_mat) <- seq_len(nrow(feature_mat))

# Creating distance matrix
dist_features <- dist(feature_mat)

return(list(
  patient_data = patient_data,
  coords_mat = coords_mat,
  dist_features = dist_features
))
}

```

Given thresholds and create binary markers

```

library(proxy)

# Threshold for the markers
thresholds <- c(
  betaCatenin = 0.253063192,
  CD11b = 1.100536222,
  CD11c = 0.213556566,
  CD138 = 1.038502303,
  CD16 = 0.304460727,
  CD20 = 0.296460556,
  CD209 = 0.406669859,
  CD3 = 0.250762424,
  CD31 = 1.006381212,
  CD4 = 0.223762,
  CD45 = 0.327450495,
  CD45RO = 0.236334141,
  CD56 = 0.770183826,
  CD63 = 0.268126101,

```

```

CD68 = 1.128510101,
CD8 = 0.260237333,
dsDNA = 0.20301697,
EGFR = 1.069264889,
FoxP3 = 0.34649899,
H3K27me3 = 0.201893232,
H3K9ac = 0.406071229,
HLA_Class_1 = 0.23773137,
HLADR = 0.4408116,
IDO = 0.312824202,
Keratin17 = 0.291843697,
Keratin6 = 0.282480556,
Ki67 = 0.210413737,
Lag3 = 1.239097091,
MPO = 0.324934545,
p53 = 0.265704293,
panKeratin = 0.244129571,
PD1 = 0.972859909,
PDL1 = 1.352024101,
pS6 = 0.282699091,
SMA = 0.250358024,
Vimentin = 0.28239596
)

# Function to replace marker values after comparing with thresholds
replace_values <- function(column, threshold) {
  ifelse(column >= threshold, 1, 0)
}

#' This function returns necessary dataframes for a given sample.
#' @param spe Spatial Experiment object containing all samples
#' @param sample_number The sample number as a string. For sample numbers less
#' than 2 digits, use 0 as a prefix. Eg: "04","09"
#' @return A list of data frames and matrices corresponding to the provided sample

get_sample_data <- function(spe, sample_number){

  # Sub-setting patient data
  spe_patient <- spe[, spe$sample_id==paste("Sample",sample_number, sep="_")]

  # Creating the feature and coordinates matrix for the selected patient
  feature_mat <- as.data.frame(t(assay(spe_patient)))

```

```

coords_mat <- as.data.frame(spatialCoords(spe_patient))

# Applying transformation based on thresholds
feature_mat <- feature_mat %>%
  mutate(across(everything(), ~replace_values(., thresholds[cur_column()])))

# Create a data frame using features and coordinates
patient_data <- cbind(feature_mat, spatialCoords(spe_patient), cells = spe_patient$mm)

# Removing gene names from the row header
rownames(feature_mat) <- seq_len(nrow(feature_mat))

# Creating distance matrix
dist_features <- dist(feature_mat, method = "Jaccard")

return(list(
  patient_data = patient_data,
  coords_mat = coords_mat,
  dist_features = dist_features
))
}

```

Modified silhouette function

```

calculate_ai <- function(i, clusters, dist_matrix) {

  # Get the current cluster for observation i
  current_cluster <- clusters[i]

  # Get the observation numbers in the same cluster as i
  current_cluster_obs <- which(clusters == current_cluster)

  # Exclude observation i from the list of current cluster observations
  current_cluster_obs <- current_cluster_obs[current_cluster_obs != i]

  if (length(current_cluster_obs) == 0) {
    return(0)
  }

  # Calculate the average distance between i and other observations in the same cluster

```

```

a_i <- mean(dist_matrix[i, current_cluster_obs])

return(a_i)
}

#' This function finds clusters that have links between them.
#'
#' @param clusters A vector containing the clusters of the data points.
#' @param neighborhood_matrix The binary neighborhood matrix.
#' @return(neighboring_clusters) A list containing neighboring clusters for
#' each cluster.
#'
calculate_neighboring_clusters <- function(clusters, neighborhood_matrix) {

  # Get unique cluster labels
  unique_clusters <- unique(clusters)

  # Initialize an empty list to store neighboring clusters for each cluster
  neighboring_clusters <- list()

  # For each cluster, find neighboring clusters based on the neighborhood matrix
  for (cluster in unique_clusters) {
    cluster_obs <- which(clusters == cluster)
    neighboring_clusters[[as.character(cluster)]] <- unique(
      clusters[
        (colSums(neighborhood_matrix[cluster_obs, , drop = FALSE]) > 0) &
        (clusters != cluster)
      ]
    )
  }

  return(neighboring_clusters)
}

#' This function calculates the b(i) and the nearest neighboring cluster
#'
#' @param i A single observation.
#' @param clusters A vector containing the clusters of the data points.
#' @param dist_matrix The distance matrix of the features.
#' @param neighboring_clusters A list containing neighboring clusters for each
#' clusters.

```

```

#' @return A list containing the nearest neighboring cluster and b(i)
#'
calculate_bi_nb <- function(i, clusters, dist_matrix, neighboring_clusters) {

  # Get the current cluster for observation i
  current_cluster <- clusters[i]

  # Get precomputed neighboring clusters for the current cluster
  neighbors <- neighboring_clusters[[as.character(current_cluster)]]

  # If no neighboring clusters, return Inf for b_i
  if (length(neighbors) == 0) {
    return(list(neighbor = NA, b_i = Inf))
  }

  # Calculate the average distance to observations in neighboring clusters
  b_i_values <- sapply(neighbors, function(cluster) {
    cluster_points <- which(clusters == cluster)
    mean(dist_matrix[i, cluster_points])
  })

  # Get the smallest average distance (min b_i) and corresponding neighboring cluster
  min_b_i <- min(b_i_values)
  neighbor <- neighbors[which.min(b_i_values)]

  return(list(neighbor = neighbor, b_i = min_b_i))
}

#' This function calculates the modified silhouette score for a given set of clusters
#'
#' @param clusters A vector containing the clusters of the data points.
#' @param dist_matrix The distance matrix of the features.
#' @param neighborhood_matrix The binary neighborhood matrix.
#' @return(wds) A silhouette class object.

calculate_modified_silhouette <- function(clusters, dist_matrix, neighborhood_matrix) {

  # Convert dist object to matrix
  dist_matrix <- as.matrix(dist_matrix)

  # Precompute neighboring clusters for each cluster
  neighboring_clusters <- calculate_neighboring_clusters(clusters, neighborhood_matrix)

```

```

# Sequential computation of silhouette widths and neighbors
results <- lapply(seq_len(length(clusters)), function(i) {
  a_i <- calculate_ai(i, clusters, dist_matrix)
  b_i_nb <- calculate_bi_nb(i, clusters, dist_matrix, neighboring_clusters)
  sil_width <- (b_i_nb$b_i - a_i) / max(a_i, b_i_nb$b_i)

  if (is.nan(sil_width)){
    sil_width = 1
  }

  # Return silhouette width and neighbor
  list(sil_width = sil_width, neighbor = b_i_nb$neighbor)
})

# Extract silhouette widths and neighbors from results
sil_widths <- sapply(results, function(res) res$sil_width)
neighbors <- sapply(results, function(res) res$neighbor)

# Combine results into a matrix for output
wds <- cbind(
  cluster = clusters,
  neighbor = neighbors,
  sil_width = sil_widths
)

attr(wds, "Ordered") <- FALSE
attr(wds, "call") <- match.call()
class(wds) <- "silhouette"

return(wds)
}

```

Obtain optimal number of clusters and neighborhood size

```

evaluate_modified_silhouette <- function(
  num_clusters, num_neighbors,
  linkage = c("ward.D2", "single", "complete", "average"),
  dist_features, coords_mat, plot = TRUE){

  if (is.null(linkage) || length(linkage) != 1L || !linkage %in% c("ward.D2", "single", "complete", "average")){
    stop("linkage must be one of 'ward.D2', 'single', 'complete', or 'average'")
  }

```

```

    stop("'linkage' argument must be one of: 'Ward.D2','single', 'complete', 'avereage'")
  }

# Obtain the linkage method.
linkage <- match.arg(linkage)

# Pre-allocate a list store the average modified silhouette score for each combination
results <- vector("list", length(num_clusters)*length(num_neighbors))

counter = 1

for (knn in num_neighbors) {

  # Create new a new neighborhood structure.
  nb <- knn2nb(knearneigh(coords_mat, k = knn)) %>%
    nb2listw(style = "W")

  # Creating the data frame needed for the constr.hclust function.
  neighbors <- listw2sn(nb)[,1:2]

  # Constraint hierarchical clustering
  constr.clust <- constr.hclust(
    d = dist_features, method = linkage,
    links = neighbors, coords = coords_mat
  )

  for (i in num_clusters) {

    # Cut the tree to obtain different number of clusters
    clusters <- cutree(constr.clust, k = i)

    # Apply the modified silhouette score method
    sil_modified <- calculate_modified_silhouette(
      clusters, dist_features, listw2mat(nb)
    )

    results[[counter]] <-list(
      num_neighbors = knn,
      num_clusters = i,
      avg_modified_sil = mean(sil_modified[,3])
    )
    counter <- counter + 1
  }
}

```

```

    if (plot) {
      plot(sil_modified, main = paste(i, "Clusters with", knn, "neighbors"), border = NA)
    }
  }
}
return(as.data.frame(bind_rows(results)))
}

```

MMD² with inverse multiquadric kernel

```

compute_mmd_sq <- function(
  sample1_idx, sample2_idx, dist_matrix,
  kernel = c("Gaussian", "IMQ"), kernel_param = 1
) {

  if (!kernel %in% c("Gaussian", "IMQ")){
    stop("'kernel' must be one of: 'Gaussian','IMQ'")
  }

  N <- length(sample1_idx)
  M <- length(sample2_idx)

  # Obtain the kernel function.
  kernel = match.arg(kernel)

  if (kernel == "Gaussian") {
    # First term: 1/N^2 sum K(xn, xn')
    term1 <- sum(exp(-dist_matrix[sample1_idx, sample1_idx]^2/kernel_param)) / (N * N)

    # Second term: -2/NM sum K(xn, xm')
    term2 <- -2 * sum(exp(-dist_matrix[sample1_idx, sample2_idx]^2/kernel_param)) / (N * M)

    # Third term: 1/M^2 sum K(x'm, x'm')
    term3 <- sum(exp(-dist_matrix[sample2_idx, sample2_idx]^2/kernel_param)) / (M * M)

    # MMD^2 value
    mmd_sq <- term1 + term2 + term3
  }
  else{
    # First term: 1/N^2 sum K(xn, xn')

```

```

    term1 <- sum(1/(sqrt(kernel_param+(dist_matrix[sample1_idx, sample1_idx])^2))) / (N * N)

    # Second term: -2/NM sum K(xn, xm')
    term2 <- -2 * sum(1/(sqrt(kernel_param+(dist_matrix[sample1_idx, sample2_idx])^2))) / (N * M)

    # Third term: 1/M^2 sum K(x'm, x'm')
    term3 <- sum(1/(sqrt(kernel_param+(dist_matrix[sample2_idx, sample2_idx])^2))) / (M * M)

    # MMD^2 value
    mmd_sq <- term1 + term2 + term3
  }

  return(mmd_sq)
}

#' This function permute labels between two clusters and calculates the MMD^2 for
#' the permuted data.
#'
#' @param sample1_idx Indices of cluster 1 observations aligning with the
#' indices of dist_matrix. This is a vector.
#' @param sample2_idx Indices of cluster 2 observations aligning with the
#' indices of dist_matrix. This is a vector.
#' @param dist_matrix Distance matrix of the features.
#' @param patient_data Data set containing features, original cluster labels (regions)
#' and polygon_ids.
#' @return(perm_mmd_sq) Numerical value.
#'
compute_perm_mmd_sq <- function(
  sample1_idx, sample2_idx,
  dist_matrix, patient_data,
  kernel = "IMQ", kernel_param = 1
) {

  # Saving the original order.
  patient_data$point <- 1:nrow(patient_data)

  #Store the indices to a list and sort it based on size.
  indices <- list(sample1_idx, sample2_idx)
  indices <- indices[order(sapply(indices, length))]

  # Extract data points from the dataset.(data from the smaller cluster is at the top)
  subset_data <- patient_data[c(indices[[1]], indices[[2]]),]

```

```

# Creating a block_id column to have a unique id for each polygon.
subset_data <- subset_data |>
  group_by(region, polygon_id) |>
  mutate(block_id = cur_group_id()) |>
  ungroup()

# Size of the smaller cluster
n_size <- sum(subset_data$region == unique(subset_data$region)[1])

# To store the sampled blocks and the count of observations sampled.
blk_ids <- c()
n_permuted <- 0

while (n_permuted < n_size) {
  new_block <- sample(
    setdiff(
      unique(subset_data$block_id),
      blk_ids),
    1
  )
  blk_ids <- c(blk_ids, new_block)

  # The number of data points in the sampled blocks.
  n_permuted <- length(subset_data$polygon_id[subset_data$block_id %in% blk_ids])
}

# Store the permuted cluster label in permuted_region column.
subset_data <- subset_data |>
  mutate(
    permuted_region = ifelse(
      block_id %in% blk_ids,
      unique(subset_data$region)[1],
      unique(subset_data$region)[2]
    )
  )

# Computing the MMD^2 for the permuted data
perm_mmd_sq <- compute_mmd_sq(
  subset_data$point[subset_data$permuted_region == unique(subset_data$permuted_region)[1]],
  subset_data$point[subset_data$permuted_region == unique(subset_data$permuted_region)[2]],
  dist_matrix, kernel, kernel_param
)

```

```

    return(perm_mmd_sq)
}

#' This function conducts 2 sample MMD^2 nonparametric test.
#'
#' @param sample1_idx Indices of cluster 1 observations aligning with the
#' indices of dist_matrix. This is a vector.
#' @param sample2_idx Indices of cluster 2 observations aligning with the
#' indices of dist_matrix. This is a vector.
#' @param dist_matrix Distance matrix of the features.
#' @param patient_data Data set containing features, original cluster labels (regions)
#' and polygon_ids.
#' @param kernel Kernel function for calculating MMD.
#' @param kernel_param Parameter value required for the kernel function.
#' @param nperm Number of permutations.
#' @return(two_sample_list) A list containing observed MMD^2, p-value and null distribution.
#'
two_sample_mmd <- function(
  sample1_idx, sample2_idx,
  dist_matrix, patient_data,
  kernel = "IMQ", kernel_param = 1, nperm = 200){

  # Observed MMD values
  obs_mmd_sq <- compute_mmd_sq(sample1_idx, sample2_idx, dist_matrix, kernel, kernel_param)

  # Distribution of MMDs under the null hypothesis
  null_mmd_sq <- lapply(
    1:nperm,
    function(i){
      compute_perm_mmd_sq(sample1_idx, sample2_idx, dist_matrix, patient_data, kernel, kernel_param)
    }
  ) |> unlist()

  p_value <- mean(null_mmd_sq >= obs_mmd_sq)

  two_sample_result <- list(obs_mmd_sq = obs_mmd_sq, p_value = p_value, null_dist = null_mmd_sq)

  return(two_sample_result)
}

#' This functions conducts multiple hypothesis testing

```

```

#'
#' @param patient_data Data set containing features, original cluster labels (regions)
#' and polygon_ids.
#' @param dist_matrix Distance matrix of the features.
#' @param kernel Kernel function for calculating MMD.
#' @param kernel_param Parameter value required for the kernel function.
#' @param nperm Number of permutations.
#' @param adj_p Adjusted p-value method
#' @return(results_summary) A data frame consisting of all pairwise test results.
#'
multiple_comparison <- function(
  patient_data, dist_matrix, kernel = "IMQ",
  kernel_param = 1, nperm = 200, adj_p = c("BH", "bonferroni", "holm")
){

  adj_p <- match.arg(adj_p)

  # list to store pairwise results
  pairwise_results <- list()

  # Unique cluster pairs
  cluster_pairs <- combn(unique(patient_data$region), 2)

  # Conduct 2 sample test for each pair
  for (i in 1:ncol(cluster_pairs)) {

    cluster_1 <- which(patient_data$region == cluster_pairs[1, i])
    cluster_2 <- which(patient_data$region == cluster_pairs[2, i])

    pairwise_results[[i]] <- c(
      list(clusters = c(cluster_pairs[1, i], cluster_pairs[2, i])),
      two_sample_mmd(
        cluster_1, cluster_2, dist_matrix, patient_data,
        kernel, kernel_param, nperm
      )
    )
  }

  # Create a dataframe of all multiple comparison results
  results_summary <- map_dfr(pairwise_results, ~ data.frame(

```

```

    region_1 = .x$clusters[1],
    region_2 = .x$clusters[2],
    obs_mmd_sq = .x$obs_mmd_sq,
    p_value = .x$p_value,
    null_dist = l(list(.x$null_dist))
  ))

# Add an adjusted p-value column for needed for multiple hypothesis testing
results_summary$adj_p <- p.adjust(results_summary$p_value, method = adj_p)

return(results_summary)
}

```

Create blocks within clusters

```

create_blocks <- function(patient_data, num_features, knn) {

  # Initialize an empty list to blocks of each region
  blk_data <- list()
  regions <- unique(patient_data$region)

  # Store the initial order of the rows
  patient_data$idx <- seq(1:nrow(patient_data))

  # For each region perform kmeans blocking
  for (i in 1:length(regions)) {

    region_data <- patient_data[patient_data$region == regions[i],]

    # Extracting the features for kmeans
    df <- region_data[,1:num_features]

    # Number of blocks based on the cluster size
    num_blks <- floor(nrow(df)/knn)

    # If the cluster is smaller than the neighborhood size or the number of
    # distinct rows is less than the number of blocks, use 1 block

    num_blks <- ifelse(num_blks == 0 | num_blks >= nrow(unique(df)), 1, num_blks)
  }
}

```

```

    if (num_blks == 1) {
      region_data$polygon_id = 1
    }
    else{

      kmeans_result <- kmeans(df, num_blks)
      region_data$polygon_id = kmeans_result$cluster
    }

    blk_data[[i]] <- region_data
  }

  combined_sf_df <- do.call(rbind, blk_data)

  # Re-arrange the dataframe to the original order
  combined_sf_df <- combined_sf_df[order(combined_sf_df$idx),]

  # Remove the idx column
  combined_sf_df <- combined_sf_df %>% select(-idx)

  return(combined_sf_df)
}

```

Pairwise test

```

pairwise_results_to_matrix <- function(df, plot = TRUE){

  link <- ifelse(df$adj_p >= 0.05, 1, 0)
  df$link <- link
  all_nodes <- unique(c(df$region_1, df$region_2))
  edges <- df[df$link == 1, c("region_1", "region_2", "obs_mmd_sq")]

  graph <- graph_from_data_frame(
    d = edges, vertices = data.frame(name = all_nodes), directed = FALSE
  )

  result_matrix <- as_adjacency_matrix(graph, attr = "obs_mmd_sq", sparse = FALSE)

  num_nodes <- vcount(graph)
}

```

```

layout <- layout_in_circle(graph)

if (plot) {
  plot(
    graph,
    layout = layout,
    edge.label = round(E(graph)$obs_mmd_sq,2), # Display weights on edges
    vertex.size = 30,
    vertex.label.cex = 1.5,
    edge.curved = 0.1,
    edge.width = 2,
    edge.color = "lightgrey",
    vertex.color = "white",
    vertex.label.color = "black"
  )
}

return(result_matrix)
}

```